

SENTINEL-X

Dossier technique et fonctionnel

Un boîtier de surveillance autonome et un centre de commandement local pour protéger les micro-centrales AetherCorp contre l'intrusion, les risques environnementaux et les cyberattaques.

Consortium FORTEX · Groupe G[n]

Kephren (DEV) · Mamadou SECK (IA & Data) · Ismahel (INFRA) · [Nom] (CYBER)

Code source : github.com/Fortexworkshop/ia · Option technique B (PC Serveur Local)

Sommaire

1. Idée et objectifs
2. Démarche et organisation
3. Architecture et choix techniques
4. Schéma du réseau
5. Schéma de câblage électronique
6. Boîtier virtuel et démonstration
7. Supervision : dashboard et Grafana
8. Matrice de sécurité
9. Documentation de l'IA
10. Rapport d'audit de sécurité
11. Points forts
12. Difficultés, limites et améliorations
13. Installation et exploitation
14. Annexe : poster A3

1. Idée et objectifs

En 2050, AetherCorp exploite des micro-centrales dans des zones isolées, sans personnel sur place. Trois menaces pèsent sur elles en même temps : l'**intrusion physique** (espionnage industriel), les **risques environnementaux** (surchauffe, fuite de gaz) et les **cyberattaques**. SENTINEL-X répond aux trois avec un boîtier Edge et un PC Serveur Local durcis.

Objectifs

Objectif	Réponse apportée	Indicateur
Détecter une intrusion	Vision YOLOv8 + liste blanche faciale ; alarme si une personne reste 20 s non reconnue	alarme physique (buzzer, LED) + alerte
Anticiper un incident	IA prédictive Isolation Forest sur séries temporelles, sans seuil statique	50/50 incidents, 436 s d'avance moyenne
Sécuriser les flux	MQTTs TLS 1.3, comptes et ACL, jetons séparés, conteneurs durcis	7/7 contrôles réussis
Superviser	Dashboard React temps réel ; Grafana pour l'exploitation (MCO)	mesure affichée en moins d'une seconde

50/50

incidents détectés

436 s

d'avance sur le seuil

7/7

contrôles sécurité

114

tests automatisés

2. Démarche et organisation

L'équipe fonctionne comme un studio d'ingénierie à quatre filières. Chaque brique a d'abord été développée dans son dépôt (`dev` , `infra` , `ia`), puis intégrée en suivant des **contrats d'interface** fixés ensemble : topics MQTT, format JSON des mesures et des alertes, routes de l'API.

Jour	Travail réalisé
Lundi	Cahier des charges, choix de l'option B, architecture réseau et flux de données
Mardi	Conteneurs (Mosquitto, PostgreSQL), dashboard, premiers modèles IA (vision, Isolation Forest)
Mercredi	Intégration Edge-to-Server : backend, MQTTS, dashboard branché ; boîtier virtuel faute de capteurs
Jeudi	Gel du code, durcissement (OWASP, comptes MQTT, jetons), supervision Grafana, audit
Vendredi	Soutenance et démonstration

Git : commits sémantiques (Conventional Commits), branches de fonctionnalité fusionnées par pull request. Les trois dépôts sont regroupés dans `Forteworkshop/ia` avec leur historique (`git subtree`), pour un seul lancement.

3. Architecture et choix techniques



Brique	Choix	Justification
Boîtier	ESP8266, C++ (PlatformIO), boîtier virtuel	firmware compilé et prêt ; capteurs physiques non disponibles dans l'équipe
Transport	MQTT (Mosquitto) en TLS	protocole léger pour l'IoT, chiffré, droits par compte
Serveur	FastAPI, PostgreSQL, Docker Compose	API et WebSocket réactifs, stack reproductible
IA	YOLOv8n, OpenCV (YuNet, SFace), MediaPipe, scikit-learn	temps réel sur CPU, modèles explicables
Supervision	React (Vite), Grafana, Prometheus	opérateur d'un côté, exploitation (MCO) de l'autre
Hors Docker	vision, IA prédictive, backend	Docker Desktop sous Windows n'accède pas à la webcam USB

Contrats d'interface

Capteurs : ESP8266 vers MQTT, topic `sentinel/<node_id>/sensors` , toutes les 2 s :

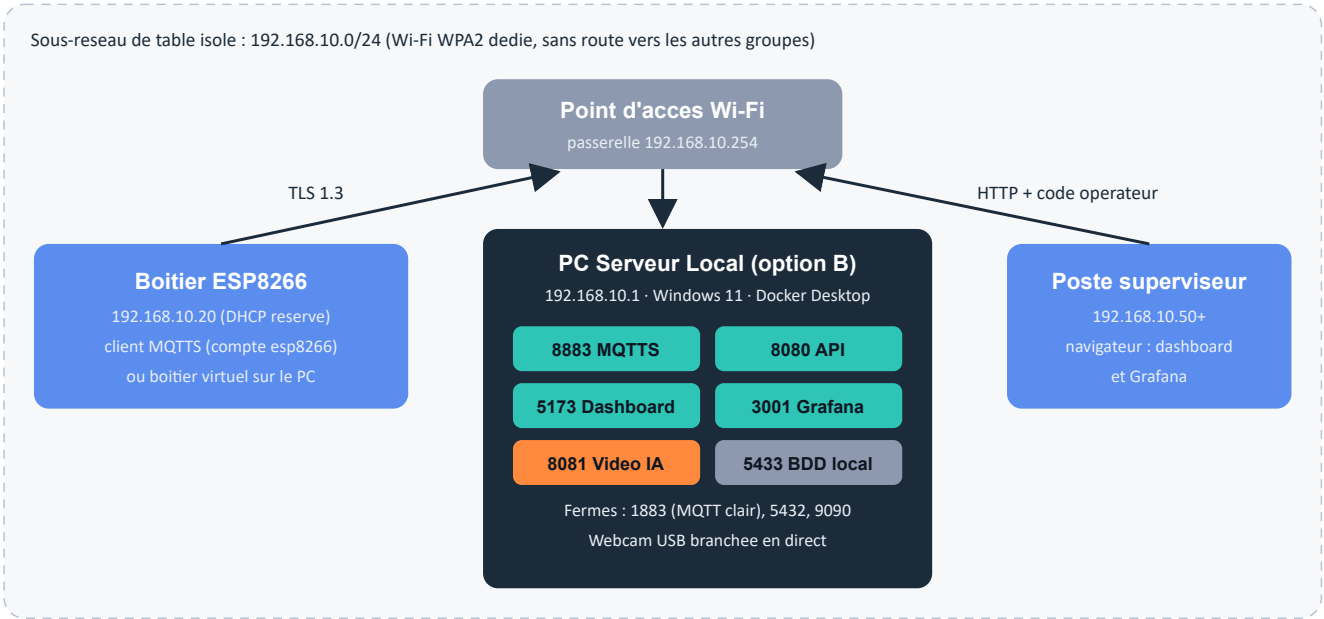
```
{"node_id": "SENTINEL-X-01", "temperature": 23.4, "humidity": 45.1, "gas": 312, "pir": 0}
```

Alertes : IA vers API, `POST /api/v1/alerts` , en-tete `Authorization: Bearer <token>` :

```
{
  "node_id": "SENTINEL-X-01",
  "source": "vision",
  "type": "INTRUSION",
  "severity": "critical",
  "message": "Presence humaine suspecte detectee (1 personne(s))",
  "timestamp": "2026-10-06T14:32:05+02:00",
  "data": {"persons": 1, "intruders": 1, "max_confidence": 0.87, "latency_ms": 54.2}
}
```

Pour `source = "anomaly"` , `type = "ENV_ANOMALY"` et `data` contient `score` , `reading` et `features` (les indicateurs de la fenetre, utiles pour expliquer l'alerte sur le dashboard).

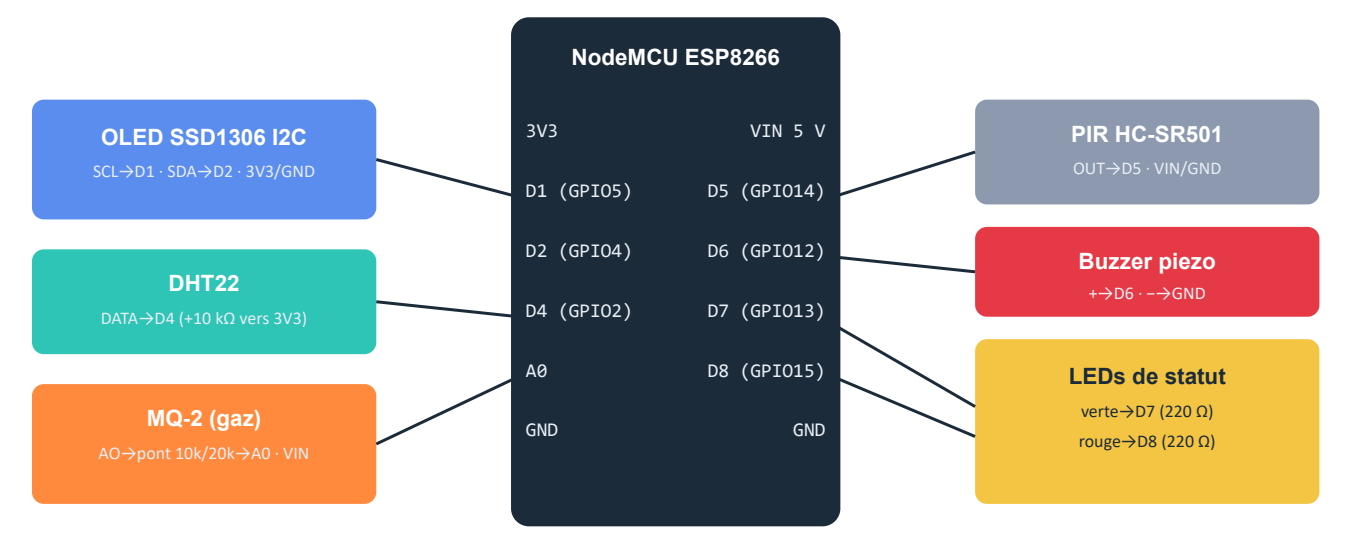
4. Schéma du réseau



Équipement	Adresse	Rôle
Point d'accès Wi-Fi de la table	192.168.10.254	réseau isolé, WPA2, sans route vers les autres groupes
PC Serveur Local	192.168.10.1	Docker (MQTT, PostgreSQL, Grafana), backend, IA, dashboard
Boîtier ESP8266	192.168.10.20	publie les mesures en MQTTS ; reçoit les commandes
Postes superviseurs	192.168.10.50 et plus	navigateur : dashboard et Grafana

À adapter au plan d'adressage réellement configuré sur la table (équipe INFRA). Le certificat TLS est émis pour 192.168.10.1 (`infra/scripts/gen-certs.sh <IP>` pour une autre adresse).

5. Schéma de câblage électronique



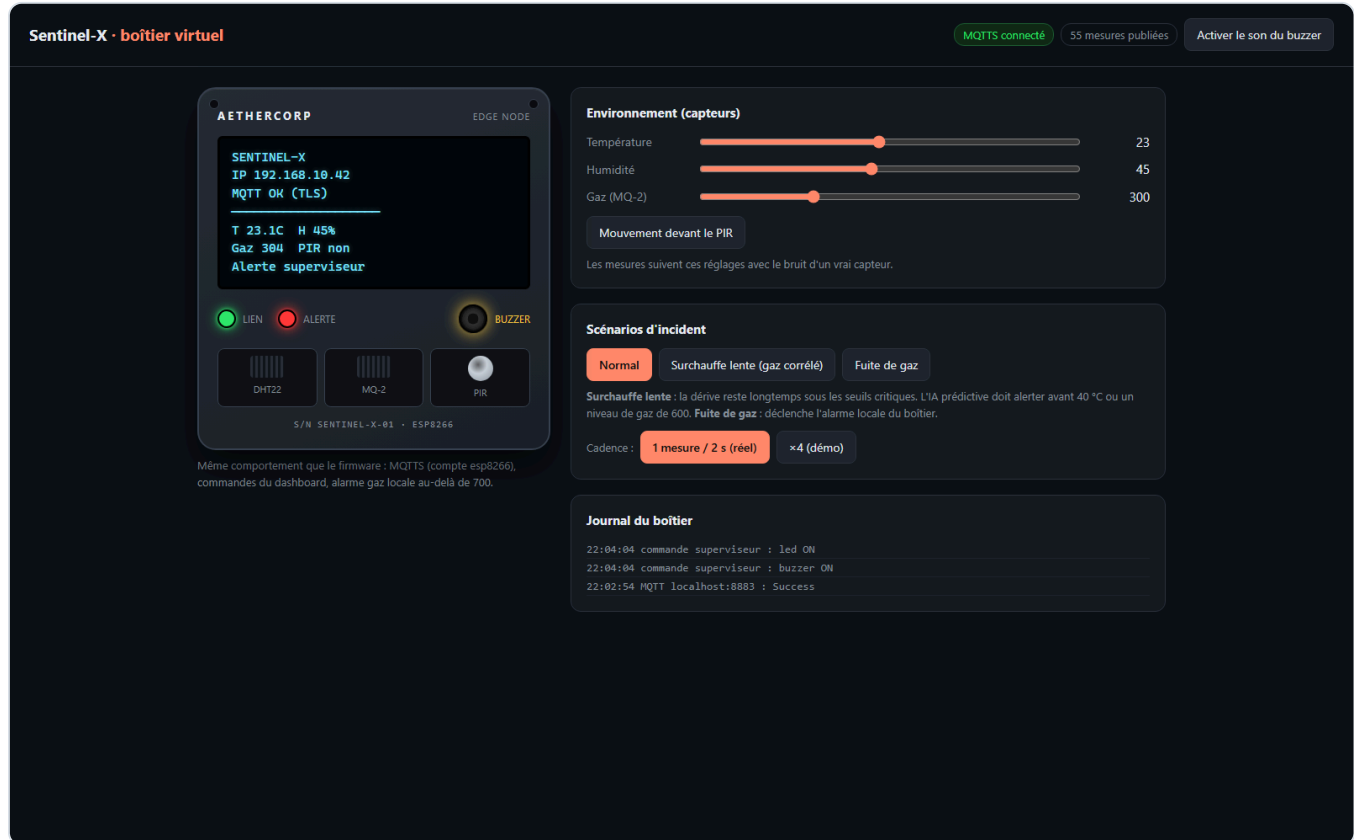
Câblage

Composant	Broche du composant	NodeMCU	GPIO
DHT22	DATA (+ résistance 10 kΩ vers 3V3)	D4	2
DHT22	VCC / GND	3V3 / GND	
MQ-2	AO (pont diviseur : 5 V vers 3,3 V max, par exemple 10 kΩ + 20 kΩ)	A0	ADC
MQ-2	VCC / GND	VIN (5 V) / GND	
PIR HC-SR501	OUT	D5	14
PIR HC-SR501	VCC / GND	VIN (5 V) / GND	
Buzzer piézo	+ (– vers GND)	D6	12
LED verte	anode (via résistance 220 Ω)	D7	13
LED rouge	anode (via résistance 220 Ω)	D8	15
OLED SSD1306 0,96" I2C	SDA / SCL	D2 / D1	4 / 5
OLED SSD1306	VCC / GND	3V3 / GND	

L'entrée A0 du NodeMCU accepte au maximum 3,3 V : ne pas brancher la sortie 5 V du MQ-2 en direct. Le MQ-2 a besoin d'environ 1 minute de préchauffage avant des valeurs stables.

6. Boîtier virtuel et démonstration

Faute de capteurs physiques, le **boîtier virtuel** (`scripts/virtual_esp.py`) remplace l'ESP8266 avec le **même protocole** que le firmware : compte MQTTS `esp8266` , mêmes topics, mêmes commandes, même alarme gaz locale. Les mesures ont le bruit d'un vrai DHT22 et d'un vrai MQ-2. Des scénarios d'incident servent à la démonstration.

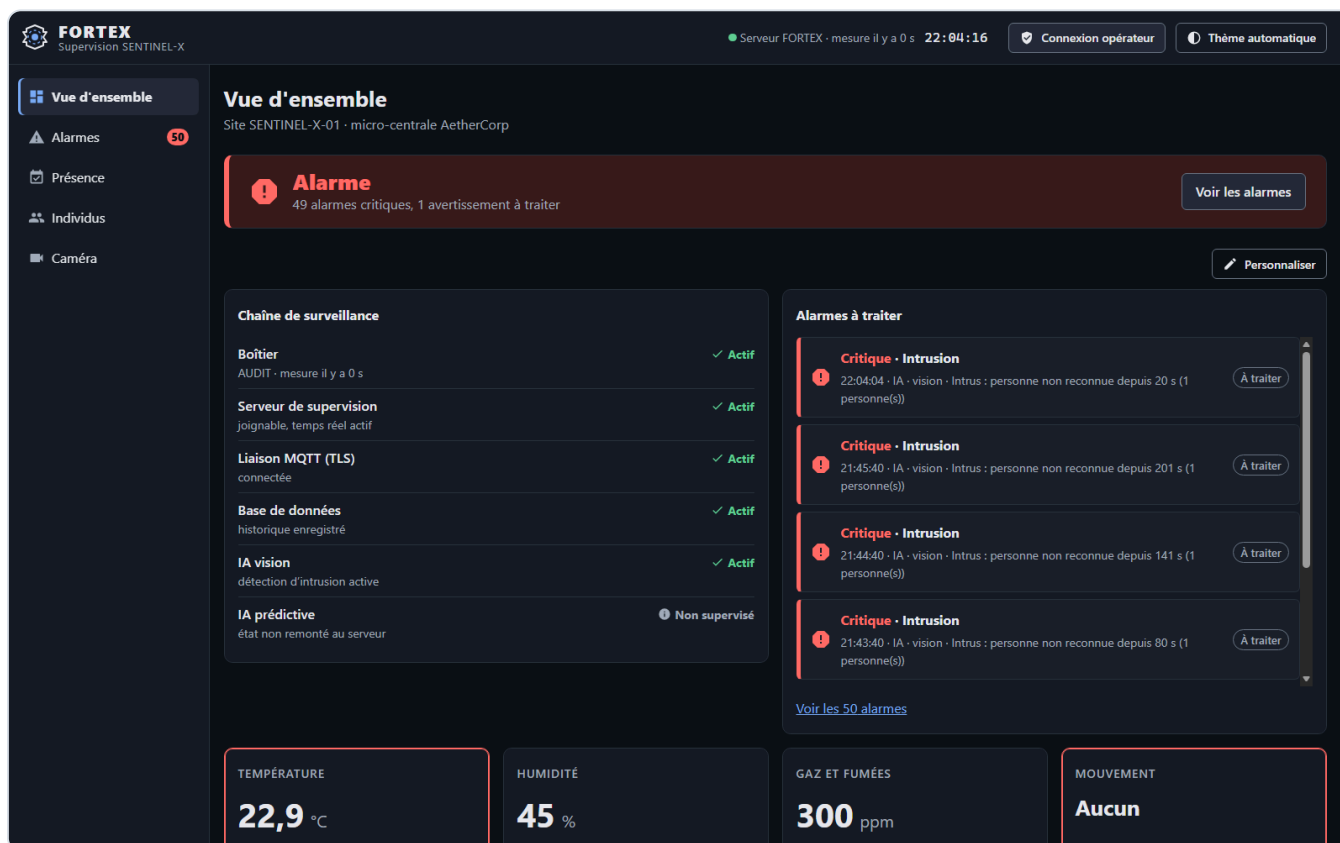


Boîtier virtuel : écran OLED, LEDs, buzzer, réglage des capteurs et scénarios (surchauffe lente, fuite de gaz).

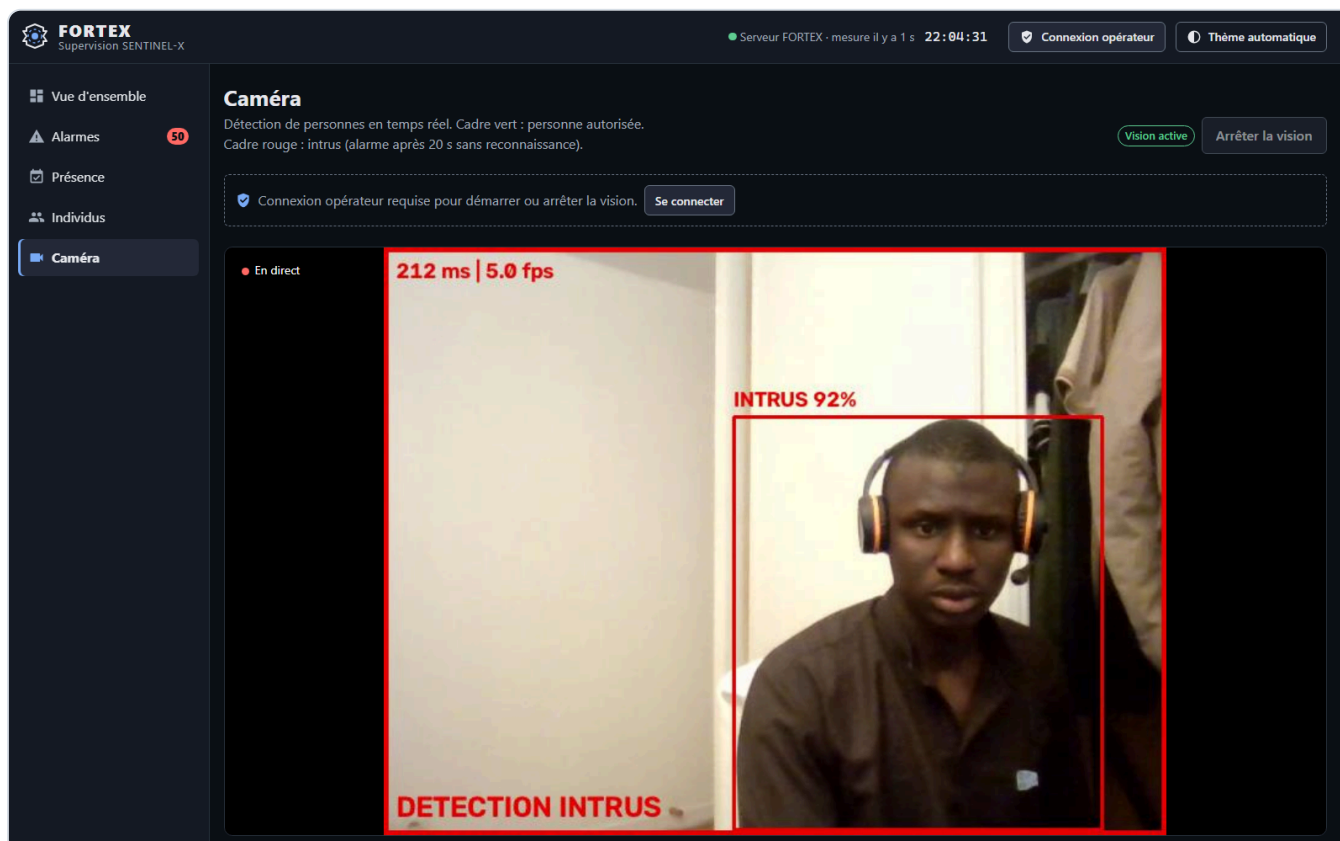
Scénario de démonstration

1. **Surchauffe lente** : l'IA prédictive alerte vers 25 °C, bien avant le seuil de 40 °C.
2. **Personne autorisée** devant la caméra : cadre vert, nom et rôle, aucune alarme.
3. **Intrus** : non reconnu pendant 20 s, d'où une alerte sur le dashboard et le buzzer et la LED rouge du boîtier.
4. **Superviseur** : il coupe l'alarme depuis le dashboard et suit l'historique dans Grafana.

7. Supervision : dashboard et Grafana



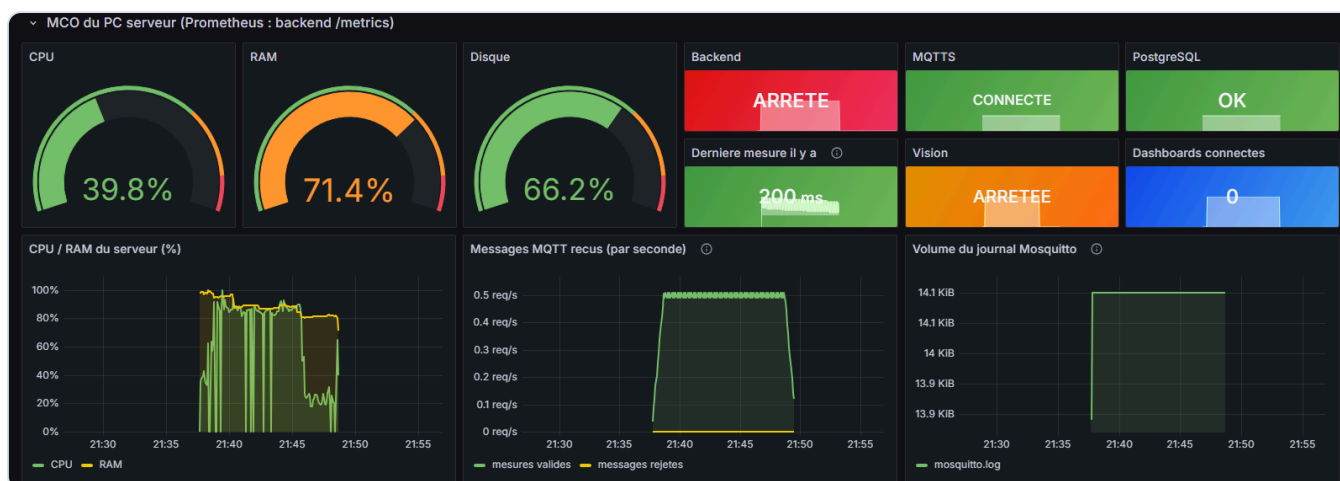
Dashbord opérateur (React) : chaîne de surveillance, alarmes à traiter, capteurs en temps réel.



Page Caméra : flux de la vision IA. Ici, une personne non reconnue est signalée comme intrus.



Grafana : capteurs et alertes depuis PostgreSQL (compte en lecture seule).



Grafana, section MCO : CPU, RAM, disque, état des services, débit MQTT, volume des logs Mosquitto.

8. Matrice de sécurité

Consortium FORTEX. Contrôles vérifiés automatiquement par `python scripts/check_security.py`.

1. Chiffrement des flux

Flux	Avant	Après	Preuve
ESP8266 → broker	MQTT en clair, port 1883	MQTTS, TLS 1.2 minimum , port 8883. Le certificat serveur est vérifié par l'ESP8266 (BearSSL, CA FORTEX embarquée)	Écran OLED « MQTT OK (TLS) » ; logs Mosquitto <code>negotiated TLSv1.3</code>
Backend, IA → broker	MQTT en clair	MQTTS , vérification du certificat par la CA FORTEX	<code>check_security.py</code> : TLSv1.3, TLS_AES_256_GCM_SHA384
IA → API	HTTP sans authentification	HTTP + jeton Bearer (secret dans <code>.env</code> , hors Git)	Requête sans jeton : HTTP 401
Backend → PostgreSQL	—	Publié sur localhost uniquement (<code>127.0.0.1:5433</code>), injoignable depuis le réseau	<code>docker compose ps</code> : <code>127.0.0.1:5433->5432/tcp</code>
Grafana → PostgreSQL	—	Compte grafana_ro en lecture seule (SELECT uniquement)	<code>infra/postgres/init/02-grafana-readonly.sh</code>

PKI : `infra/scripts/gen-certs.sh` crée une autorité racine « FORTEX Root CA » et un certificat serveur RSA 2048 valable 2 ans. Il couvre les noms `mosquitto`, `localhost` et l'adresse IP du PC serveur. La clé privée de la CA reste sur le PC serveur ; seul `ca.crt` est distribué.

2. Authentification et droits (moindre privilège)

Compte MQTT	Peut publier	Peut lire	Usage
<code>esp8266</code>	<code>sentinel/+/sensors</code> , <code>sentinel/+/status</code> , <code>fortex/capteurs/mesures</code>	<code>sentinel/+/commands</code>	Boîtier, simulateur
<code>serveur</code>	<code>sentinel/+/commands</code>	<code>sentinel/+/sensors</code> , <code>sentinel/+/status</code> , <code>fortex/capteurs/mesures</code>	Backend, IA
<code>anonyme</code>	refusé	refusé	—

Les mots de passe sont aléatoires (`gen-mqtt-users.sh`) et hachés dans le fichier `passwd`. Un compte volé reste limité : le compte `serveur` **ne peut pas injecter de fausses mesures**, ce qui a été vérifié.

3. Durcissement des conteneurs (`infra/docker-compose.yml`)

Mesure	Services	Effet
<code>no-new-privileges</code>	tous	Aucune élévation de privilèges dans le conteneur
Utilisateur non-root (UID 10001)	backend	Une faille de l'API ne donne pas root
<code>cap_drop: ALL</code>	backend, simulateur	Aucune capacité Linux
<code>read_only: true</code> + <code>tmpfs /tmp</code>	backend, simulateur	Système de fichiers non modifiable

Mesure	Services	Effet
Rotation des logs (10 Mo × 3)	tous	MCO : le flux MQTT continu ne remplit pas le disque
Healthchecks	postgres, backend	Redémarrage et ordre de démarrage fiables
Limites Mosquitto (max_connections 50 , message_size_limit 4096 , max_queued_messages 1000)	broker	Atténue un déni de service pendant le pentest croisé

4. Exposition réseau du PC serveur

Port	Service	Exposé
8883	MQTTS	réseau de la table (ESP8266)
8080	API backend	réseau de la table (dashboard)
5173	dashboard	réseau de la table
8081	flux vidéo IA	réseau de la table
5000	contrôle d'accès (biométrie)	localhost uniquement
5433	PostgreSQL	localhost uniquement (127.0.0.1), pour le backend du PC serveur
3001	Grafana (supervision, MCO)	réseau de la table, compte admin obligatoire
1883	MQTT en clair	fermé

Pare-feu Windows, à appliquer en PowerShell administrateur. Ces règles autorisent uniquement le sous-réseau de la table :

```
$table = "192.168.10.0/24"
New-NetFirewallRule -DisplayName "FORTEX MQTTS" -Direction Inbound -Protocol TCP -LocalPort 8883 -
RemoteAddress $table -Action Allow
New-NetFirewallRule -DisplayName "FORTEX API" -Direction Inbound -Protocol TCP -LocalPort 8080,5173,8081
-RemoteAddress $table -Action Allow
New-NetFirewallRule -DisplayName "FORTEX bloque MQTT clair" -Direction Inbound -Protocol TCP -LocalPort
1883 -Action Block
```

5. Secrets

- Aucun secret dans Git : les `.env`, `certs/`, `mqtt-users.env`, `passwd` et `firmware/.../sentinel_config.h` sont exclus par les `.gitignore`.
- Les fichiers `*.example` documentent chaque variable, avec des valeurs factices.

6. Données personnelles (biométrie)

Seules les empreintes faciales sont conservées (128 nombres par agent), jamais les photos : une photo envoyée depuis le dashboard ou la plateforme est convertie en empreinte puis oubliée. Le traitement est 100 % local, et la plateforme n'est accessible que depuis le PC serveur.

La liste est modifiable depuis le dashboard (page « Individus ») ou en ligne de commande (`scripts/enroll.py`) : les deux écrivent dans le même `data/faces.npz`, que la vision `--whitelist` recharge à **chaud** (contrôle toutes les 2 s) : un individu supprimé n'est plus reconnu. La suppression (`DELETE /api/v1/people/{name}`) efface les données et l'empreinte (droit à l'effacement).

7. Application web : OWASP Top 10:2025 (dashboard et API)

Catégorie	Risque identifié	Mesure	Preuve
A01 Contrôle d'accès	Données personnelles (individus, présence) et actions (acquiescement, exercice) ouvertes à tout le réseau	Code opérateur exigé ; présence diffusée par WebSocket aux seuls clients authentifiés	<code>tests/test_backend.py</code> : accès anonyme = 401 ; WebSocket anonyme sans présence
A02 Configuration	Pas d'en-têtes de sécurité ; serveur de développement Vite pendant la démo ; CORS * par défaut	En-têtes <code>nosniff</code> , <code>X-Frame-Options</code> , <code>no-store</code> sur l'API ; CSP stricte (empreintes SHA-256, ni <code>unsafe-inline</code> ni <code>eval</code>) ; démo sur build de production (<code>pnpm run demo</code>) ; aucune origine CORS par défaut	<code>test_security_headers_and_no_wildcard_cors</code> ; 0 violation CSP au navigateur
A03 Chaîne d'approvisionnement	Vite ≤ 6.4.2 : 4 vulnérabilités dont 1 élevée (contournement <code>server.fs.deny</code> sous Windows)	Vite 6.4.4, lockfile pnpm, scripts d'installation limités à esbuild	<code>pnpm audit</code> : aucune vulnérabilité connue
A04 Cryptographie	Dashboard et API en HTTP sur le réseau de la table : le code opérateur circule en clair	— (voir § 8)	—
A05 Injection	Textes saisis (noms, consignes, messages)	React échappe tout rendu ; requêtes SQL paramétrées ; CSP sans script en ligne	revue de code
A06 Conception	Arrêt de la vision sans garde-fou ; photo sans taille maximale	Confirmation explicite et état « Vigilance » tant que la vision est arrêtée ; photo ≤ 8 Mo	<code>test_photo_size_is_bounded</code>
A07 Authentification	Code opérateur compilé dans le JavaScript (lisible par tous)	Code saisi par l'opérateur, gardé dans l'onglet (sessionStorage), jamais dans le bundle	contrôle automatique : code absent des scripts servis
A08 Intégrité	Messages temps réel et disposition locale acceptés sans contrôle	Validation de chaque message WebSocket ; disposition relue et bornée	message malformé ignoré sans plantage
A09 Journalisation	Aucune trace des actions de l'opérateur	<code>data/audit.log</code> : acquiescement, commande, exercice, vision, individus, échec d'authentification WebSocket	journal lu après test
A10 Conditions exceptionnelles	Échecs d'action silencieux (console) ; une erreur de	Chaque échec est affiché à l'opérateur ; acquiescement refusé = alarme rétablie ;	tests navigateur

Catégorie	Risque identifié	Mesure	Preuve
	composant efface l'écran	un widget en erreur n'affecte pas les autres	

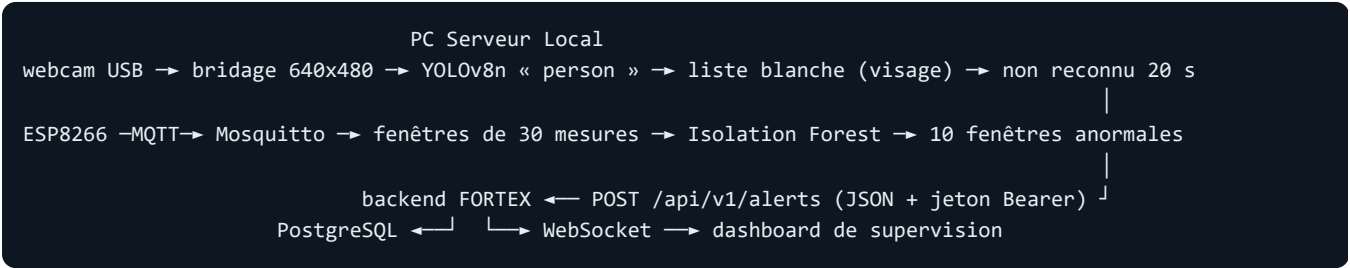
8. Reste à la charge de l'équipe CYBER

- HTTPS pour l'API et le dashboard : reverse proxy TLS, avec la même CA (OWASP A04 : le code opérateur circule aujourd'hui en clair sur le réseau de la table).
- Flux vidéo (:8081/video) servi sans authentification : toute machine du réseau de la table voit la caméra. À protéger (jeton ou passage par l'API).
- Comptes opérateurs nominatifs : le code opérateur est partagé, le journal d'audit ne peut donc attribuer une action qu'à un poste (adresse IP), pas à une personne.
- IA prédictive : aucun signal de vie vers le serveur. Si `sentinel_anomaly.py` s'arrête, rien ne le signale (le dashboard l'indique « Non supervisé »).
- Accès SSH par clé uniquement (si le serveur est sous Linux ou WSL), et appliquer les règles de pare-feu ci-dessus.
- Pentest croisé du jeudi (Nmap, Wireshark, Metasploit) et rapport d'audit. `check_security.py` fournit la base de l'auto-audit.

9. Documentation de l'IA

Consortium FORTEX — Workshop EPSI Bac+4, octobre 2026. Dépôt : [Fortexworkshop/ia](#) .

La brique IA protège la micro-centrale AetherCorp contre deux des trois menaces du cahier des charges : **l'intrusion physique** (vision par ordinateur sur la webcam USB) et **les risques environnementaux** (surchauffe, fuite de gaz) par maintenance prédictive. Tout s'exécute en local sur le PC Serveur Local (option B) : aucune image ni mesure ne quitte la table.



1. Vision intelligente : détection d'intrusion

Script : `scripts/sentinel_vision.py` (module `sentinel/vision.py`).

Étape	Choix	Justification
Capture	Webcam USB branchée sur le PC serveur, OpenCV	Exigé par le sujet
Bridge	Redimensionnement systématique en 640x480	Exigé par le sujet, coût de calcul constant
Modèle	YOLOv8n (Ultralytics), pré-entraîné sur COCO, filtré sur la classe <code>person</code>	Plus petit modèle YOLOv8, temps réel sur CPU
Taille d'inférence	480 px	Meilleur compromis mesuré (voir tableau)
Seuil de confiance	0,5	Écarte les détections douteuses
Alarme intrusion	Personne non reconnue pendant 20 s (perte de moins de 2 s tolérée), puis rappel toutes les 60 s ; l'alerte allume buzzer et LED du boîtier	Un passage bref ou un visage tourné une seconde ne déclenche rien ; un agent reconnu jamais
Liste blanche	Visage reconnu (YuNet + SFace, similarité cosinus $\geq 0,363$) dans la boîte de la personne	Un technicien autorisé n'est pas un intrus
Sorties	Alerte <code>INTRUSION</code> (gravité <code>critical</code>), flux vidéo annoté MJPEG <code>:8081/video</code>	Dashboard et historique

Performance mesurée (exigence : < 100 ms par trame)

Mesure sur la webcam du PC serveur, CPU uniquement, 40 images par taille (`python scripts/bench_vision.py`). Latence = bridge + inférence.

Taille d'inférence	Moyenne	95e centile	Images/s	< 100 ms
640	74 ms	124 ms	13,5	non
480 (retenue)	32 ms	34 ms	31,4	oui
416	26 ms	28 ms	38,2	oui

Taille d'inférence	Moyenne	95e centile	Images/s	< 100 ms
320	22 ms	23 ms	46,2	oui

La latence de chaque image est affichée en direct sur le flux, en vert sous 100 ms et en rouge au-dessus, pour le prouver pendant la démo.

2. Maintenance prédictive : anomalies cinétiques

Scripts : `scripts/sentinel_train.py` (entraînement), `scripts/sentinel_anomaly.py` (temps réel). Module : `sentinel/anomaly.py`.

Pourquoi pas de seuil statique

Le sujet interdit `if temp > 40`. Un seuil ne voit l'incident qu'une fois qu'il est **déjà** là. L'exemple du sujet, une hausse lente de température corrélée à une micro-déviations du gaz, reste longtemps sous tout seuil raisonnable. On détecte donc **un comportement anormal**, pas une valeur anormale.

Données

Le DHT22 (température, humidité) et le MQ-2 (gaz, valeur analogique 0–1023) sont publiés par l'ESP8266 toutes les 2 s sur `sentinel/<id>/sensors`.

Indicateurs (9 par fenêtre glissante de 30 mesures, soit 1 minute)

Capteur	Indicateurs
Température	moyenne, écart-type, pente (régression linéaire)
Humidité	moyenne, pente
Gaz	moyenne, écart-type, pente
Croisé	corrélation température / gaz

Les pentes et la corrélation captent la dynamique (« cinétique ») : une dérive lente et simultanée des deux capteurs ne ressemble à aucune fenêtre normale.

Modèle

- Normalisation standard (`StandardScaler`), puis **Isolation Forest** (Scikit-Learn, 200 arbres).
- Apprentissage **non supervisé**, sur le régime normal uniquement : il n'est pas nécessaire d'avoir vécu un incident pour l'entraîner.
- Seuil appris, pas fixé à la main** : une fenêtre est anormale si elle est plus atypique que la plus atypique des fenêtres normales vues à l'entraînement.
- Confirmation** : l'alerte part après **10 fenêtres anormales consécutives** (20 s). La gravité (`warning` ou `critical`) dépend de l'écart au seuil.
- Explicabilité** : l'alerte contient les 9 indicateurs de la fenêtre. Le dashboard peut afficher *pourquoi* l'IA a alerté (par exemple une pente du gaz de +0,63 par mesure et une corrélation température/gaz de 0,77).

Entraînement

- Avant le boîtier** : 12 régimes normaux simulés de 600 mesures (`sentinel/simulate.py`), avec niveau de base, oscillation lente, bruit des capteurs et pics isolés. Les fenêtres ne chevauchent jamais deux séries, sinon une coupure créerait une fausse pente apprise comme « normale ».
- Avec le boîtier** : enregistrement d'environ 10 minutes en régime normal (`sentinel_anomaly.py --record`), puis réentraînement sur ces mesures réelles (`sentinel_train.py --csv data/sensors.csv`). Le modèle apprend ainsi le niveau réel de la salle.

Résultats

Évaluation sur des scénarios simulés indépendants de l'entraînement : 50 incidents (surchauffe de +0,04 °C par mesure avec dérive du gaz) et 50 séries normales de 30 minutes. Commande : `python scripts/sentinel_train.py --runs 50`.

Indicateur	Résultat
Incidents détectés	50 / 50
Séries normales avec fausse alerte	1 / 50
Avance sur le seuil critique (40 °C ou gaz 600)	436 s en moyenne, 388 s au minimum

L'IA prévient donc **plus de 6 minutes avant** qu'une alarme à seuil ne sonne.

Validation sur le boîtier virtuel (`sentinel/virtual_box.py`), qui remplace les capteurs physiques, mesurée sur 20 boîtiers avec un bruit de capteur différent à chaque fois :

Cas	Résultat
Fonctionnement normal, 30 minutes par boîtier	0 fausse alerte sur 20
Scénario « surchauffe lente »	20 détections sur 20, vers 25 °C et un gaz à environ 340, alors que les seuils critiques sont à 40 °C et 600

Le backend conserve un **garde-fou de dernier recours** (`SafetyAlarm`) qui sonne quand le seuil critique est *déjà* atteint. Il est distinct de l'IA et sert à garder le site protégé si le service d'IA s'arrête.

3. Contrôle d'accès du personnel (innovation)

Plateforme : `scripts/sentinel_admin.py` , sur `http://localhost:5000`.

- **Liste blanche** : on enregistre un agent (technicien, garde) à partir de 1 à 10 photos. Seule l'empreinte faciale est conservée (128 nombres par agent), jamais les photos. La vision utilise cette liste pour distinguer un agent autorisé d'un intrus.
- **Registre des présences sur site** : l'agent est reconnu à son visage, puis un geste du pouce (MediaPipe Hand Landmarker, 21 points de la main) enregistre l'heure. Pouce en haut = entrée, pouce sur le côté = pause (aller, puis retour), pouce en bas = sortie. Export CSV possible.
- **RGPD** : ce sont des données biométriques (article 9). Le traitement est 100 % local et chaque agent peut être supprimé de la liste. Pour la démo, seuls des membres de l'équipe volontaires sont enregistrés.

4. Intégration avec les autres briques

Flux	Protocole	Format
ESP8266 → IA, backend	MQTT <code>sentinel/<id>/sensors</code>	<code>{"temperature", "humidity", "gas", "pir"}</code>
IA → backend	POST <code>/api/v1/alerts</code> , jeton Bearer	<code>{node_id, source, type, severity, message, timestamp, data}</code>
Backend → dashboard	WebSocket <code>/ws</code>	<code>reading , alert , command-ack</code>
Vision → dashboard	HTTP MJPEG <code>:8081/video</code>	flux annoté

Le secret (jeton de l'API) reste dans `.env` , exclu de Git.

5. Qualité et reproductibilité

- **114 tests automatisés** (`python -m pytest -q`) : classification des gestes, anti faux positifs, extraction des indicateurs, détection d'un incident avant le seuil critique, client d'alertes, backend (API, WebSocket, PostgreSQL en mémoire), plateforme web.
- Installation et lancement en une commande : `scripts/setup.ps1` , `scripts/start.ps1` .
- Toutes les mesures de ce document se reproduisent avec `bench_vision.py` et `sentinel_train.py` .

6. Limites et pistes

Limite	Piste
Pas de capteurs physiques : modèle entraîné et validé sur des données simulées (boîtier virtuel)	Avec du matériel : réentraîner sur les mesures réelles (<code>--record</code>)
YOLOv8n pré-entraîné, non spécialisé (affiche ou écran montrant une personne = détection)	Fine-tuning sur images du site ; zone d'intérêt
Reconnaissance faciale sensible à l'éclairage et à l'angle	3 à 5 photos variées par agent
Flux MQTT et vidéo encore en clair	MQTTS (port 8883) et HTTPS : code déjà prêt, certificats à fournir par CYBER
Vision hors Docker (accès webcam USB sous Windows)	Conteneur avec accès matériel sur Linux / Raspberry Pi

10. Rapport d'audit de sécurité

Auto-audit automatisé

Le script `scripts/check_security.py` attaque la stack réelle. Résultat sur la stack lancée :

```
[OK ] Port MQTT en clair 1883 ferme
[OK ] Connexion MQTTS anonyme refusee (Not authorized)
[OK ] Mauvais mot de passe refuse (Not authorized)
[OK ] Certificat du broker valide et session chiffree (TLSv1.3, TLS_AES_256_GCM_SHA384, emis par FORTEX Root CA)
[OK ] Compte boitier : mesure chiffree recue par le backend
[OK ] ACL : le compte serveur ne peut pas publier de fausses mesures
[OK ] POST /api/v1/alerts sans jeton refuse (HTTP 401)
7/7 controles reussis
```

Les 114 tests automatisés couvrent en plus les en-têtes de sécurité, l'absence de CORS ouvert, le refus d'accès anonyme aux données personnelles et la taille maximale des photos.

Pentest croisé (jeudi)

À compléter par l'équipe CYBER après le pentest croisé :

Attaque tentée	Outil	Cible	Résultat	Correctif
Scan de ports	Nmap	PC serveur	[]	[]
Écoute du trafic (MitM)	Wireshark	MQTT, HTTP	[]	[]
Injection de mesures	mosquitto_pub	broker	[]	[]
Déni de service	[]	broker, API	[]	[]

11. Points forts

- **Une chaîne complète et chiffrée**, du capteur à l'alarme physique, vérifiée de bout en bout.
- **Une IA qui anticipe** : alerte en moyenne 7 minutes avant le seuil critique, sans aucun seuil statique.
- **Une vision qui distingue** agent autorisé et intrus, avec un délai de 20 s pour éviter les fausses alarmes.
- **Une sécurité prouvée** : 7/7 contrôles, comptes et droits par appareil, OWASP Top 10 traité côté web.
- **Une exploitation suivie** : Grafana pour le MCO (CPU, RAM, logs MQTT).
- **Reproductible** : un dépôt, une commande d'installation, 114 tests automatisés.

12. Difficultés, limites et améliorations

Difficulté ou limite	Réponse ou piste
Pas de capteurs physiques	Boîtier virtuel au protocole identique ; flasher le firmware dès réception du matériel et réentraîner l'IA sur les vraies mesures
Latence vision au-delà de 100 ms avec reconnaissance faciale et pointage, sur un PC chargé	32 ms avec YOLO seul ; reconnaissance faciale une image sur N, modèle quantifié ou GPU
Reconnaissance faciale sensible à l'éclairage et aux accessoires (casque)	3 à 5 photos d'enregistrement dans les conditions réelles
Webcam inaccessible depuis Docker sous Windows	vision et backend sur le PC ; conteneur avec accès matériel sous Linux
Dashboard et API en HTTP	reverse proxy HTTPS avec la CA FORTEX
Intégration de trois dépôts	contrats d'interface, puis regroupement en un dépôt (git subtree)

13. Installation et exploitation

```
git clone https://github.com/Fortexworkshop/ia.git fortex
cd fortex
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1 -ServerIp 192.168.10.1
powershell -ExecutionPolicy Bypass -File scripts\start.ps1
```

Adresse	Contenu
http://localhost:5173	Dashboard de supervision (connexion opérateur pour les actions)
http://localhost:8090	Boîtier virtuel
http://localhost:3001	Grafana (supervision, MCO)
http://localhost:8080/docs	API du backend

Vérifications : `python -m pytest -q (tests)`, `python scripts/check_security.py` (sécurité), `python scripts/bench_vision.py` (latence de la vision). Aucun secret n'est versionné : `scripts/configure.py` génère les mots de passe, la CA TLS et les comptes MQTT.

14. Annexe : poster A3

Le poster au format A3 est joint en dernière page de ce dossier.

SENTINEL-X



La sécurité à la bordure

Consortium FORTEX · surveillance autonome des micro-centrales AetherCorp · Workshop EPSI Bac+4 2026



Intrusion

Une personne non reconnue reste 20 s devant la caméra : alarme, buzzer et LED.



Surchauffe, gaz

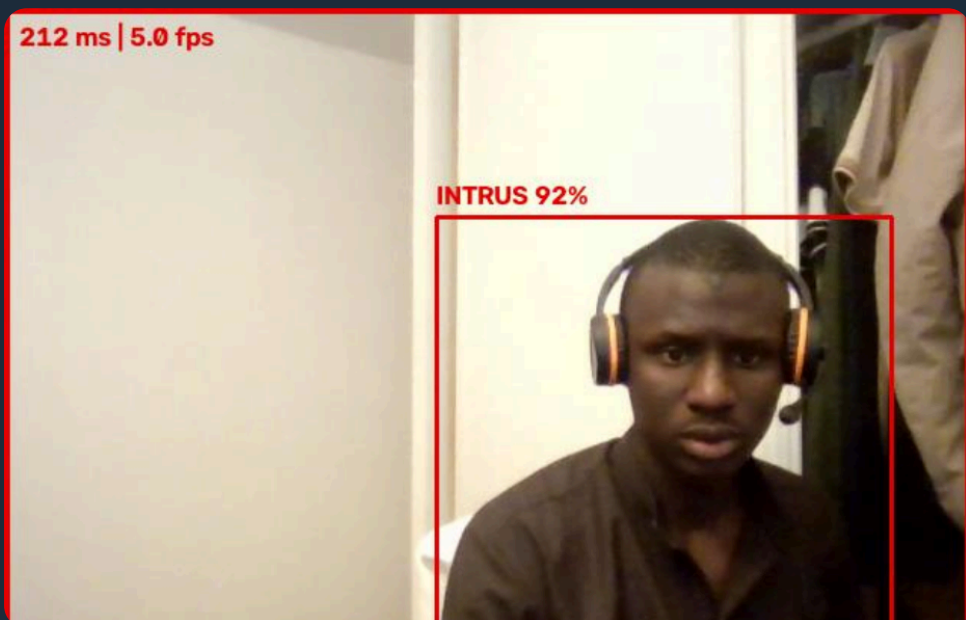
L'IA repère une dérive lente et corrélée, plusieurs minutes avant le seuil critique.



Cyberattaque

Flux chiffrés TLS 1.3, comptes et droits par appareil, conteneurs durcis.

Vision IA en direct



YOLOv8 détecte la personne, la liste blanche reconnaît les agents (nom et rôle) ; inconnu 20 s = intrus.

La chaîne complète

Boîtier ESP8266 → MQTTS → Backend

Webcam → IA vision → Alarme

Capteurs → IA prédictive → Alerte

Dashboard Grafana PostgreSQL

Boîtier virtuel au protocole identique au firmware : capteurs réglables et scénarios d'incident pour la démo.

Technologies

ESP8266 · C++ MQTT TLS FastAPI

PostgreSQL Docker Compose YOLOv8

OpenCV scikit-learn React Grafana

Prometheus

50/50

incidents détectés par l'IA prédictive

436 s

d'avance moyenne sur le seuil critique

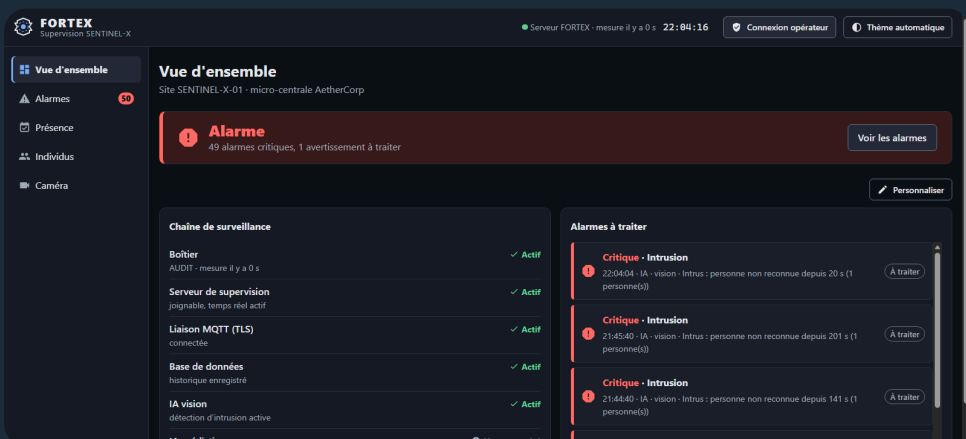
20 s

avant l'alarme pour un inconnu

7/7

contrôles de sécurité réussis

Supervision



Dashboard opérateur temps réel : alarmes, présence, individus, caméra, commandes.

Exploitation (MCO)



Grafana : capteurs, alertes, CPU, RAM, débit MQTT, logs.